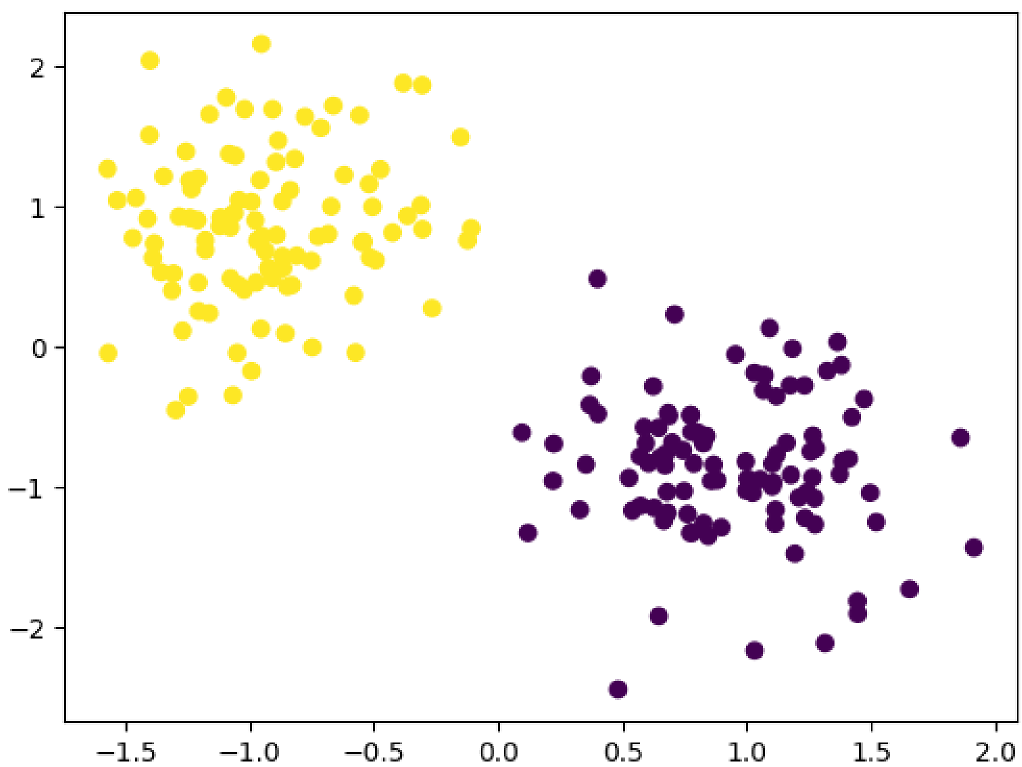
svm_E1

May 19, 2025

[1]:
```python
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
import numpy as np

def make_grid(X: np.ndarray, target_points: int = 10000) -> np.ndarray:
  """
  Create an adaptive grid of points for decision boundaries, balancing␣
↪resolution and computation time.

  Parameters:
    X (np.ndarray): Input data points.
    target_points (int): Approximate number of grid points (default: 10000).

  Returns:
    np.ndarray: Grid of points shaped (n_samples, 2).
  """
  # Calculate data range with padding
  x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
  y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

  # Handle edge cases where range is zero
  x_range = x_max - x_min
  y_range = y_max - y_min
  if x_range <= 0:
    x_range = 1e-5
    x_max = x_min + x_range
  if y_range <= 0:
    y_range = 1e-5
    y_max = y_min + y_range

  # Calculate aspect ratio and determine grid dimensions
  aspect_ratio = x_range / y_range
  nx = int(np.round(np.sqrt(target_points * aspect_ratio)))
  ny = int(np.round(np.sqrt(target_points / aspect_ratio)))
  nx, ny = max(nx, 1), max(ny, 1)  # Ensure at least 1 point per axis

  # Generate evenly spaced grid
```

```
    xx = np.linspace(x_min, x_max, nx)
    yy = np.linspace(y_min, y_max, ny)
    xx_mesh, yy_mesh = np.meshgrid(xx, yy)

    return xx_mesh, yy_mesh

X, y = make_blobs(n_samples=200, centers=2)

X = (X - X.mean(axis=0)) / X.std(axis=0)

plt.scatter(X[:, 0], X[:, 1], c=y)
plt.show()
```

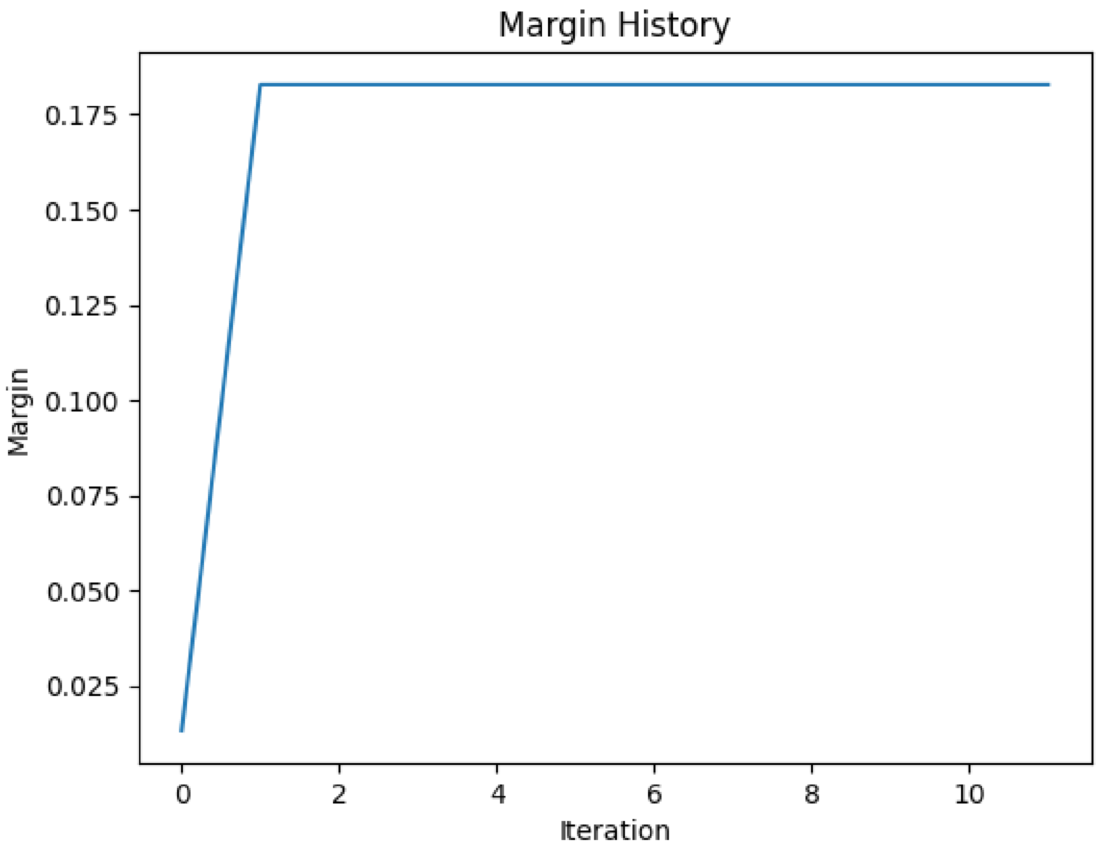


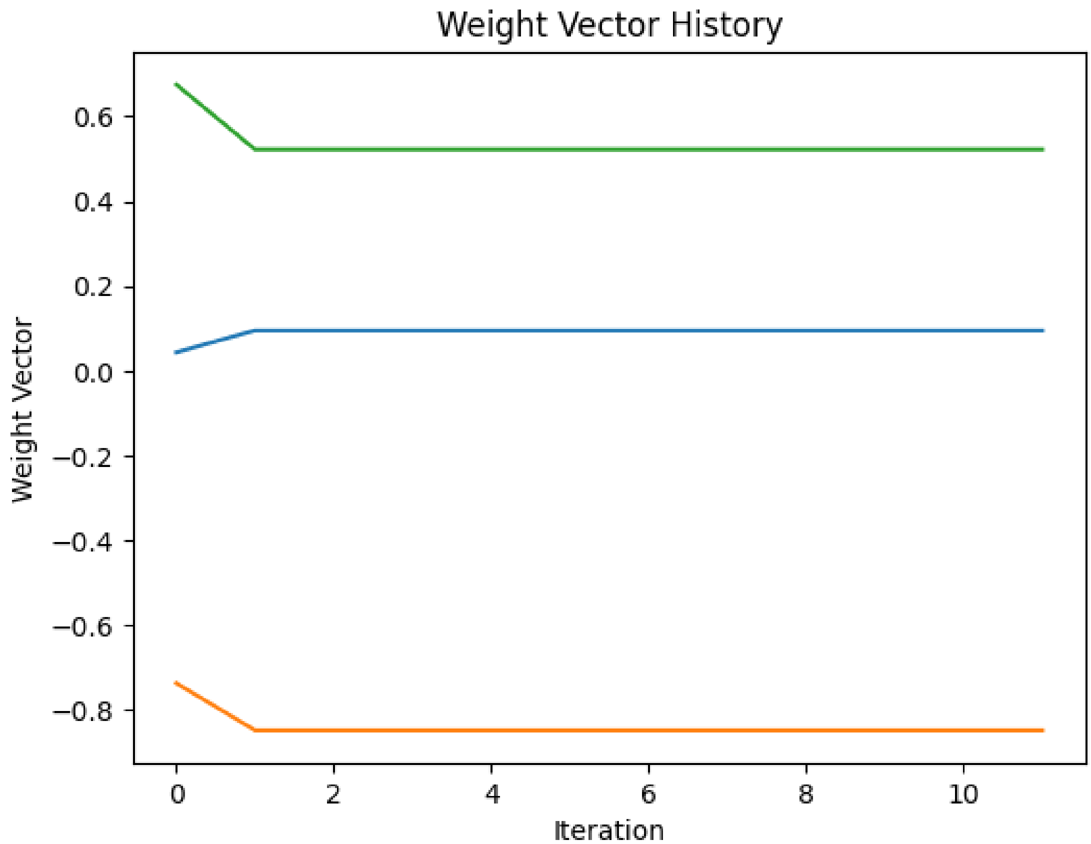
[2]:
```
y = np.where(y == 0, -1, 1)

def perceptron(X, Y, max_iter=1000, learning_rate=1.0, decay_rate=0.99, margin_threshold=0.1, patience=10):
    """
    Enhanced Perceptron with explicit margin maximization.

    Changes:
    - Margin-based weight updates
```

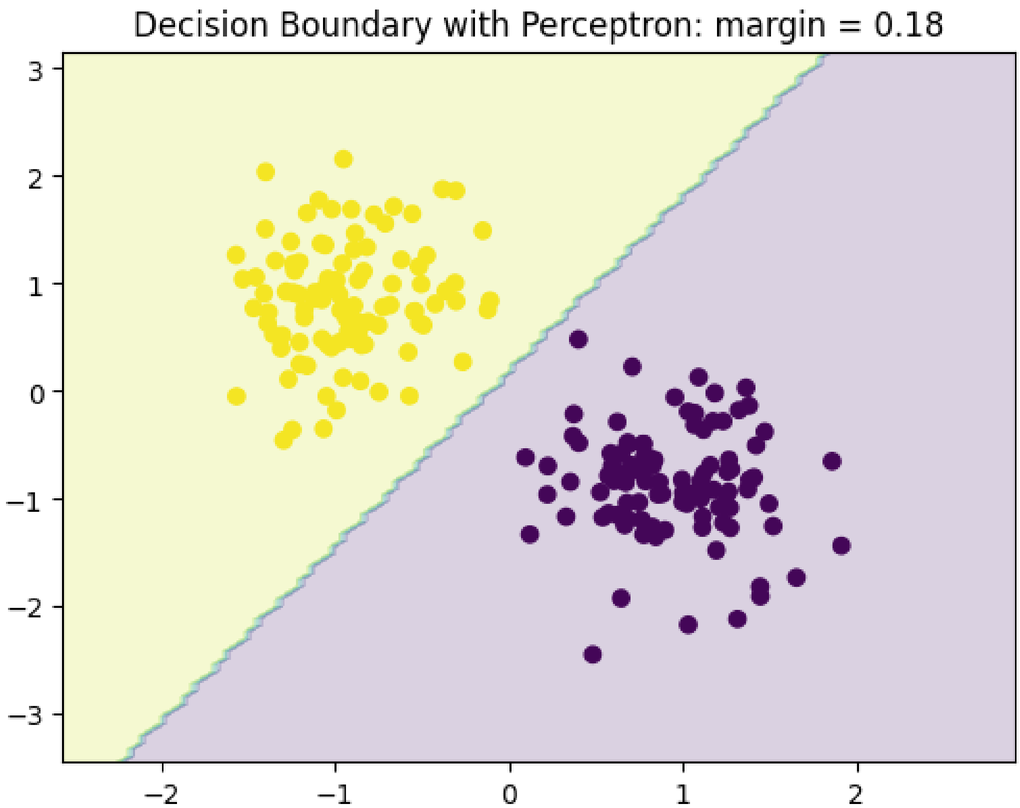
```python
    - Weight normalization
    - Adaptive learning rate reset
    - Early stopping with patience
    """
    X_augmented = np.hstack((-np.ones((X.shape[0], 1)), X))
    w = np.random.randn(X_augmented.shape[1]) * 0.01  # Random initialization
    best_w = w.copy()
    best_margin = -np.inf
    margin_history = []
    w_history = []
    no_improvement = 0  # For early stopping

    for _ in range(max_iter):
        any_updates = False
        for i in range(X_augmented.shape[0]):
            xi = X_augmented[i]
            yi = Y[i]
            score = np.dot(w, xi)

            # Margin-based update: update if within margin threshold
            if yi * score < margin_threshold:
                w += learning_rate * yi * xi / np.linalg.norm(xi)  # Normalized update
                any_updates = True

        # Normalize weights to stabilize margin calculation
        w_norm = np.linalg.norm(w)
        if w_norm > 0:
            w /= w_norm

        # Calculate margin
        scores = Y * np.dot(X_augmented, w)
        current_margin = np.min(scores)  # Already normalized

        # Track best margin
        if current_margin > best_margin:
            best_margin = current_margin
            best_w = w.copy()
            learning_rate *= 1.05  # Boost learning rate on improvement
            no_improvement = 0
        else:
            no_improvement += 1
            learning_rate *= decay_rate  # Decay otherwise

        margin_history.append(current_margin)
        w_history.append(w.copy())
```

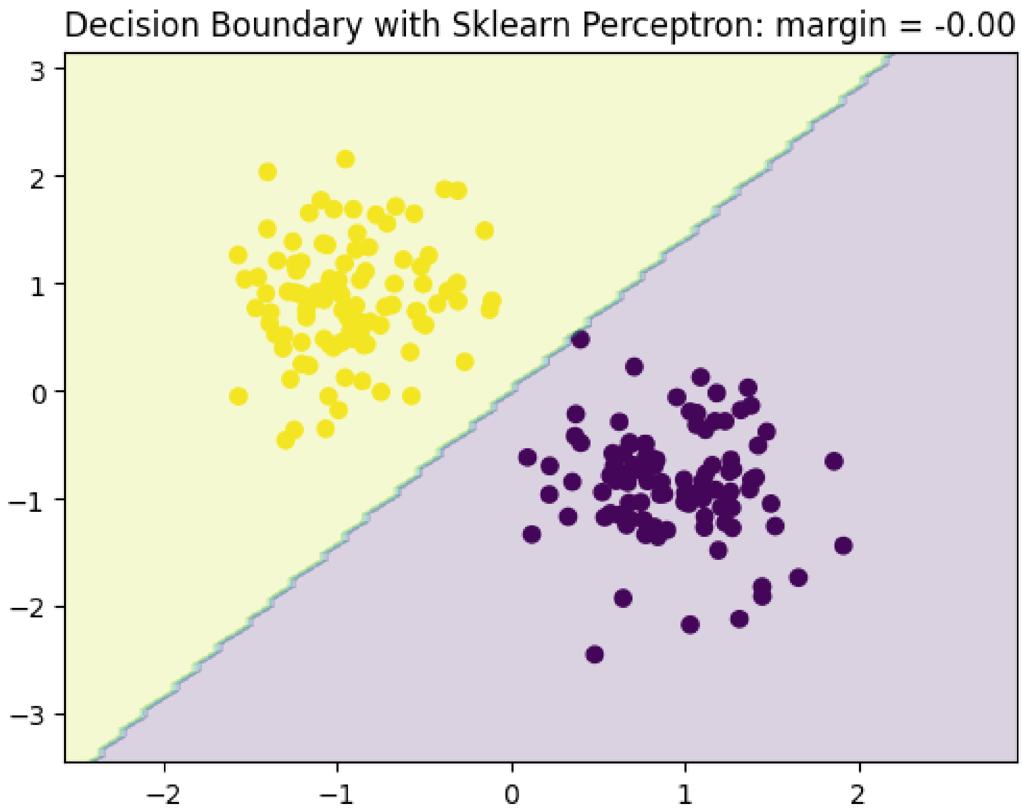
```python
        # Early stopping
        if no_improvement >= patience:
            break

    return best_w, margin_history, w_history


# Train perceptron
w, margin_history, w_history = perceptron(X, y)

# Plot margin history
plt.plot(margin_history)
plt.xlabel('Iteration')
plt.ylabel('Margin')
plt.title('Margin History')
plt.show()

# Plot weight history
plt.plot(w_history)
plt.xlabel('Iteration')
plt.ylabel('Weight Vector')
plt.title('Weight Vector History')
plt.show()

# Create grid for decision boundary
xx_mesh, yy_mesh = make_grid(X, target_points=10000)
# Predict on grid points
X_grid = np.c_[xx_mesh.ravel(), yy_mesh.ravel()]
X_grid_augmented = np.hstack((-np.ones((X_grid.shape[0], 1)), X_grid))
p = np.dot(X_grid_augmented, w)
# Reshape predictions to match grid shape
p = np.sign(p.reshape(xx_mesh.shape))

y = np.where(y == -1, 0, 1)  # Convert labels for plotting
p = np.where(p == -1, 0, 1)  # Convert predictions for plotting

# Plot decision boundary
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.contourf(xx_mesh, yy_mesh, p, alpha=0.2)
plt.xlim(xx_mesh.min(), xx_mesh.max())
plt.ylim(yy_mesh.min(), yy_mesh.max())
plt.title(f'Decision Boundary with Perceptron: margin = {margin_history[-1]:.
 ↪2f}')
plt.show()

from sklearn.linear_model import Perceptron as SklearnPerceptron
```

```
# Train sklearn perceptron
sklearn_perceptron = SklearnPerceptron(max_iter=1000, tol=1e-3, penalty='l2',␣
↪alpha=0.01)
sklearn_perceptron.fit(X, y)
# Predict on grid points
p_sklearn = sklearn_perceptron.predict(X_grid)
# Reshape predictions to match grid shape
p_sklearn = p_sklearn.reshape(xx_mesh.shape)

# Compute sklearn margin
X_biased = np.hstack((-np.ones((X.shape[0], 1)), X))
scores_sklearn = y * np.dot(X_biased, np.hstack((sklearn_perceptron.intercept_,␣
↪sklearn_perceptron.coef_.flatten())))
sklearn_margin = np.min(scores_sklearn) / np.linalg.norm(np.
↪hstack((sklearn_perceptron.intercept_, sklearn_perceptron.coef_.flatten())))

# Plot decision boundary
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.contourf(xx_mesh, yy_mesh, p_sklearn, alpha=0.2)
plt.xlim(xx_mesh.min(), xx_mesh.max())
plt.ylim(yy_mesh.min(), yy_mesh.max())
plt.title(f'Decision Boundary with Sklearn Perceptron: margin = {sklearn_margin:
↪.2f}')
plt.show()
```


Margin History
Margin
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0
2
4
6
8
10
Iteration


Weight Vector History
Weight Vector
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6
−0.8
0
2
4
6
8
10
Iteration


Decision Boundary with Perceptron: margin = 0.18
3
2
1
0
−1
−2
−3
−2
−1
0
1
2


Decision Boundary with Sklearn Perceptron: margin = -0.00
3
2
1
0
−1
−2
−3
−2
−1
0
1
2